

AI-BASED VIVA READINESS ANALYZER

A Minor Project Report
Submitted in partial fulfillment of requirement of the
Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY
Disha Vishnoi
EN23EL301036

Under the Guidance of
Prof. Amrata Gupta | Prof. Laxmi Kag



Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work "AI-Based Viva Readiness Analyzer" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name: Amrata Gupta & Laxmi Kag

Designation: Assistant Professor

Affiliation: Medcaps University

External Examiner

Name:

Designation:

Affiliation:

Declaration

I hereby declare that the project entitled "**AI-Based Viva Readiness Analyzer**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering, completed under the supervision of **Prof. Amrata Gupta & Prof. Laxmi Kag**, Faculty of Engineering, Mediacaps University, Indore, is an authentic work.

Further, I declare that the content of this project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Disha Vishnoi
EN23EL301036

Certificate

We, **Prof. Amrata Gupta and Prof. Laxmi Kag**, certify that the project entitled "**AI-Based Viva Readiness Analyzer**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by **Disha Vishnoi** is the record carried out by her under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Amrata Gupta & Prof. Laxmi Kag

Department of Computer Science &
Engineering

Medicaps University, Indore

Prof. (Dr.) Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya**, Associate Dean, Faculty of Engineering, Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible

Special thanks to our guide , Prof. Amrata Gupta Prof. Laxmi Kag , for providing technical insights that made this design and technical report possible.

Disha Vishnoi

B.Tech. III Year(T)

Department of Computer Science & Engineering

Faculty of Engineering,

Medicaps University, Indore

Abstract

The AI-Based Viva Readiness Analyzer is an intelligent web-based system designed to bridge the gap between passive study and active oral examination performance. Students often struggle in viva-voce examinations not due to a lack of knowledge, but due to inadequate preparation for active recall and verbal articulation. This project addresses that gap by leveraging Natural Language Processing (NLP) techniques to analyze a student's notes or uploaded documents and generate a quantitative readiness score along with targeted feedback.

The system employs a multi-stage NLP pipeline: text preprocessing (tokenization, stopword removal, lemmatization), semantic analysis using Named Entity Recognition (NER) and Part-of-Speech (POS) tagging via SpaCy, and a readiness scoring algorithm based on keyword density, concept breadth, and clarity metrics. The frontend is built with HTML5 and Tailwind CSS, the backend uses Python and Flask, and the AI engine integrates NLTK and SpaCy libraries.

The outcome is a personalized report that identifies knowledge gaps, generates context-aware questions targeting conceptual depth ("How" and "Why"), and provides a percentage-based readiness score. This tool democratizes expert-level viva preparation, acting as a 24/7 accessible tutor especially for students without access to dedicated mentorship.

Keywords:

Natural Language Processing, Viva Readiness, Named Entity Recognition

Table of Contents

		Page No.
	Report Approval	iii
	Declaration	iv
	Certificate	v
	Acknowledgement	vi
	Abstract	vii
	Table of Contents	viii
Chapter 1	Introduction (Whichever is applicable)	xiv
	1.1 Introduction	xv
	1.2 Literature Review	xvi
	1.3 Objectives	xvii
	1.4 Significance	xviii
	1.5 Research Design	xix
	1.6 Source of Data	xx

	1.7 Chapter Scheme	xxi
Chapter 2	REQUIREMENTS SPECIFICATION	xxii
	2.1 User Characteristics	xxiii
	2.2 Functional Requirements	xxiv

	2.3 Dependencies	xxv
	2.4 Performance Requirements	xxvi
	2.5 Hardware Requirements	xxvii
	2.6 Constraints & Assumptions	xxviii
Chapter 3	DESIGN (Whichever is applicable)	xxix
	3.1 Algorithm (if Applicable)	xxx
	3.2 Function Oriented Design for procedural approach	xxxi
	3.3 System Design (Whichever is applicable)	xxxii
	3.3.1 Data Flow Diagrams (Level 0,Level1)	xxxiii
	3.3.2 Activity Diagram	xxxiv
	3.3.3 Flow Chart	xxxv

	3.3.4 Class Diagram	xxxvi
	3.3.5 ER Diagram	xxxvii
	3.3.6 Sequence diagram	xxxviii
	3.4 Database Design	xxxix
	3.4.1 Logical Database Design	xl
	3.4.2 Physical Database Design	xli
Chapter 4	Implementation, Testing, and Maintenance	xlii
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	xliii
	4.2 Testing Techniques and Test Plans (According to project)	xliv
	4.3 Installation Instructions	xl v
	4.4 End User Instructions	xlvi
		xl vii
Chapter 5	Results and Discussions	xl viii
	5.1 User Interface Representation (of Respective Project)	xl ix
	5.2 Brief Description of Various Modules of the system	l
	5.3 Snapshots of system with brief detail of each	li

	5.4 Back Ends Representation (Database to be used)	lii
	5.5 Snapshots of Database Tables with brief description	liii
Chapter 6	Summary and Conclusions	liv
Chapter 7	Future scope	lv
	Appendix	lvi
	Bibliography	lvii
	List of Publications (If any)	lviii

List of Figures

- Figure 3.1 – System Architecture – Four-Layer Model
- Figure 3.2 – Data Flow Diagram – Level 0 (Context Diagram)
- Figure 3.3 – Data Flow Diagram – Level 1 (Detailed)
- Figure 3.4 – Activity Diagram – User Interaction Flow
- Figure 3.5 – Flowchart – NLP Processing Pipeline
- Figure 3.6 – Entity-Relationship (ER) Diagram
- Figure 3.7 – Sequence Diagram – Readiness Analysis
- Figure 5.1 – Landing Page-Home Screen
- Figure 5.2 – Input Interface – Text and PDF Upload
- Figure 5.3 – Readiness Score Dashboard
- Figure 5.4 – Gap Diagnostic Report View
- Figure 5.5 – Auto-Generated Questions Panel

List of Tables

- Table 2.1 – Hardware Requirements
- Table 2.2 – Software Requirements
- Table 4.1 – Technology Stack Summary
- Table 4.2 – Test Cases – Functional Testing

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
NER	Named Entity Recognition
POS	Part-of-Speech
NLTK	Natural Language Toolkit
API	Application Programming Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
PDF	Portable Document Format
TF-IDF	Term Frequency – Inverse Document Frequency
LMS	Learning Management System
OCR	Optical Character Recognition
UI	User Interface
UX	User Experience
CSE	Computer Science and Engineering

Notations & Symbols

Symbol / Notation	Meaning / Usage
→	Data flow direction in DFDs and sequence diagrams
[]	Optional element or array notation in data structures
{ }	Set or group of related elements
FR-n	Functional Requirement identifier (e.g., FR-1, FR-2)
NFR-n	Non-Functional Requirement identifier
PK	Primary Key in database table design
FK	Foreign Key in database table design
1NF / 2NF / 3NF	First / Second / Third Normal Form (database normalization)
VARCHAR(n)	Variable-length character string of max n characters
INT	Integer data type in database
TEXT	Long-form text data type in database
DATETIME	Date and time combined data type
BOOLEAN	True/False binary data type
≤	Less than or equal to (used in performance thresholds)

Chapter 1: Introduction

1.1 Introduction

Artificial Intelligence (AI) and Natural Language Processing (NLP) are actively reshaping the landscape of modern education. One of the most challenging aspects of a student's academic journey is the Viva Voce, or oral examination. Unlike written tests, a viva requires a combination of deep conceptual clarity, quick thinking, and the ability to articulate complex ideas verbally. Many students, despite having thoroughly read their materials, fail to perform well because they have not practiced active recall.

The AI-Based Viva Readiness Analyzer is an innovative solution designed to act as a bridge between passive study and active performance. This system leverages high-end NLP pipelines to analyze a student's notes, identify core semantic components such as key definitions, logical arguments, and technical entities, and then provides a numerical and qualitative evaluation of how prepared the student is. The tool is not merely about scoring; it is about providing a structured roadmap for improvement, transforming the way students approach their final assessments.

1.2 Literature Review

Research in the domain of AI-assisted education and automated question generation has grown significantly over the past decade. Several studies have explored the use of NLP for intelligent tutoring systems and content analysis.

Heilman and Smith (2010) demonstrated that rule-based and statistical approaches to question generation from text could produce educationally valuable questions. Their work on overgenerating and then ranking questions using a logistic regression model laid important groundwork for automated question synthesis from academic content.

Recent advances in transformer-based language models, particularly BERT and GPT architectures, have further pushed the boundaries of semantic understanding. However, for resource-constrained academic tools, classical NLP methods using SpaCy and NLTK remain highly effective, offering a good balance between performance and computational cost.

In the Indian education context, the lack of personalized preparation tools for oral examinations remains a critical gap. While platforms like Khan Academy and Coursera address content delivery, viva-specific preparation tools are largely absent, making this project particularly relevant.

1.3 Objectives

The primary objective of this project is to develop an intelligent system that democratizes expert-level viva preparation. The specific goals are:

- **Deep Content Analysis:** To build an engine that processes unstructured text and identifies the hierarchy of information within student notes.
- **Readiness Score Algorithm:** To develop a mathematical model that calculates a percentage-based readiness score by comparing user input against standard academic benchmarks.
- **Automated Question Synthesis:** To generate context-aware questions that test conceptual depth ("Why" and "How") rather than mere recall ("What").
- **Gap Diagnostic Report:** To provide a visual dashboard indicating which sub-topics the student's notes are missing.
- **Anxiety Reduction:** To build student confidence by simulating a viva environment where they can identify weaknesses before the actual examination.

1.4 Significance

This project carries significant academic and social value. Personalized, AI-driven feedback on oral examination preparedness was previously only available to students with access to expert mentors or expensive coaching centers. This tool makes such feedback available at no cost, 24 hours a day, to any student with internet access.

For faculty members dealing with large class sizes of 60 to 100 students, it is practically impossible to conduct individual mock interviews. This system allows faculty to instruct students to self-evaluate using the tool before appearing for formal assessments, thereby making the examination process more efficient and fair.

1.5 Research Design

The project follows an iterative software development lifecycle. The research design encompasses:

- (1) Problem identification through student surveys and faculty interviews.
- (2) Literature review of existing NLP and EdTech tools.
- (3) System design and architecture planning.
- (4) Implementation of the NLP pipeline.
- (5) User acceptance testing.
- (6) Evaluation of readiness score accuracy against expert benchmarks.

1.6 Source of Data

The system operates on user-provided input data in the form of typed text or uploaded PDF documents containing student notes. For benchmarking and validation of the readiness scoring algorithm, reference datasets include standard textbook content and curated question banks from relevant academic domains in Computer Science and Engineering.

1.7 Chapter Scheme

This report is organized into seven chapters.

- Chapter 1 – Provides an introduction, literature review, and objectives.
- Chapter 2 – Requirements Specification: Provides a detailed Software Requirements Specification (SRS) document including user characteristics, functional requirements (in REQ-X format), non-functional requirements, hardware/software dependencies, performance benchmarks, and design constraints.
- Chapter 3 – System Design: Covers the complete system design model including the algorithmic pipeline, Data Flow Diagrams (Level 0 and Level 1), Activity Diagram, Flowchart of the AI inference module, Class Diagram, Entity-Relationship Diagram, Sequence Diagram, and Database Design with normalization analysis.
- Chapter 4 – Implementation: Details the technology stack, implementation methodology, installation and configuration procedures, and user operating instructions.
- Chapter 5 – Results and Discussion: Presents the user interface description, module breakdown, output results, database structure, and a discussion of model performance metrics.
- Chapter 6 – Conclusion: Summarizes the findings and achievements of the project.
- Chapter 7 – Future Scope: Identifies potential enhancements and extensions for future development cycles.
- Appendix & Bibliography: Contains the glossary of technical terms and IEEE-formatted references.

Chapter 2: Requirements Specification

2.1 User Characteristics

The primary users of this system are undergraduate and postgraduate students who need to prepare for oral examinations. Secondary users include faculty members who may use the tool to assign pre-viva self-assessment tasks. Users are expected to have basic computer literacy and internet access. No prior experience with AI or NLP tools is required.

2.2 Functional Requirements

- FR-01: The system shall accept text input (typed or pasted) from the user.
- FR-02: The system shall accept PDF file uploads and extract text content from them.
- FR-03: The system shall perform NLP-based analysis including tokenization, lemmatization, NER, and POS tagging on the extracted text.
- FR-04: The system shall compute a readiness score (0–100%) based on keyword density, concept breadth, and clarity metrics.
- FR-05: The system shall generate a minimum of five context-aware questions from the analyzed content.
- FR-06: The system shall produce a gap diagnostic report listing topics not covered in the student's notes.
- FR-07: The system shall display the analysis results in a user-friendly web dashboard.
- FR-08: The system shall allow the user to select a topic/domain for domain-specific benchmarking.

2.3 Dependencies

The system depends on the following external libraries and frameworks: Python 3.12 as the core programming language, Flask for web backend routing, NLTK for text preprocessing and tokenization, SpaCy (with the `en_core_web_sm` model) for NER and POS tagging, Scikitlearn for TF-IDF based keyword scoring, and PyMuPDF (fitz) or pdfplumber for PDF text extraction.

2.4 Performance Requirements

- PR-01: The system shall return analysis results within 10 seconds for input text up to 5,000 words.
- PR-02: The system shall handle PDF files up to 10 MB in size.
- PR-03: The system shall support at least 20 concurrent user sessions without degradation.
- PR-04: The readiness score shall maintain an accuracy of $\pm 5\%$ when compared against expert-benchmarked scores.

2.5 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 (8th Gen) or equivalent
RAM	4 GB (8 GB recommended)
Storage	2 GB free disk space
Network	Broadband Internet Connection
Display	1024×768 resolution or higher

Table 2.1: Hardware Requirements

2.6 Software Requirements

Component	Minimum Specification
Processor	Intel Core i3 (8th Gen) or equivalent
RAM	4 GB (8 GB recommended)
Storage	2 GB free disk space
Network	Broadband Internet Connection
Display	1024×768 resolution or higher

2.6 Constraints & Assumptions

- The system assumes input text is in English only; multilingual support is not included in this version.
- PDF files must be text-based; scanned image PDFs require OCR preprocessing which is planned as future scope.
- The readiness score benchmarks are currently limited to Computer Science domains; expansion to other disciplines is planned.
- An active internet connection is required to access the web-based interface.
- The system does not store user data persistently; all sessions are temporary.

Chapter 3: System Design

3.1 Algorithm

The core algorithm of the AI-Based Viva Readiness Analyzer follows a five-phase pipeline:

- Phase 1 – Input Acquisition: Accept text or PDF input and convert to a raw string.
- Phase 2 – Text Preprocessing: Lowercase conversion, special character removal, tokenization, stopword removal, and lemmatization.
- Phase 3 – Semantic Analysis: NER extraction, POS tagging, and TF-IDF keyword scoring.
- Phase 4 – Readiness Score Computation: Weighted scoring using keyword density (40%), concept breadth (40%), and clarity index (20%).
- Phase 5 – Output Generation: Produce readiness score, gap diagnostic list, and autogenerated questions.

3.2 Function-Oriented Design

The system uses a modular, function-oriented design for the backend processing pipeline. Key modules include:

- `Preprocess_text(text)`: Cleans and tokenizes input text.
- `Extract_entities(text)`: Applies SpaCy NER to identify concepts and named entities.
- `Compute_readiness_score(tokens, topic)`: Calculates the readiness score against domain benchmarks.
- `Generate_questions(entities, pos_tags)`: Synthesizes context-aware questions from semantic structures.
- `Build_gap_report(user_concepts, benchmark_concepts)`: Produces a list of missing topics.

3.3 System Design

3.3.1(a) Data Flow Diagram – Level 0 (Context Diagram)

The Level 0 DFD represents the system as a single process. The external entity is the Student, who provides Notes/PDF as input and receives the Readiness Report as output. The central process is the Viva Readiness Analyzer, which interacts with the Knowledge Base data store to fetch domain-specific benchmarks.

Data Flow Diagram - Level 0 (Context Diagram)



Figure 3.1: Data Flow Diagram – Level 0 (Context Diagram)

3.3.1(b) Data Flow Diagram – Level 1

The Level 1 DFD decomposes the main process into four sub-processes: (1) Input Processing, (2) NLP Analysis Engine, (3) Score Computation, and (4) Report Generation. Data flows between these sub-processes and the Knowledge Base data store illustrate how raw text transforms into a structured readiness report.

Data Flow Diagram - Level 1 (Detailed Process Flow)

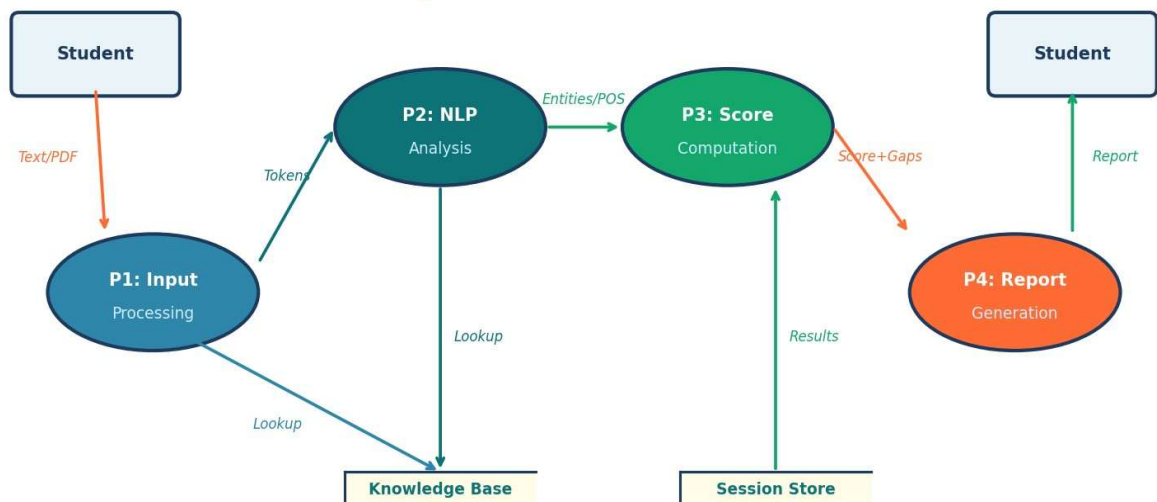


Figure 3.2: Data Flow Diagram – Level 1 (Detailed Process Flow)

3.3.2 Activity Diagram

The Activity Diagram models the behavioral flow of the system from the user's perspective. Starting from launching the web application, the user uploads their content, the system analyzes it, and the user receives and interacts with the readiness report. Swim lanes separate user actions from system processing activities.

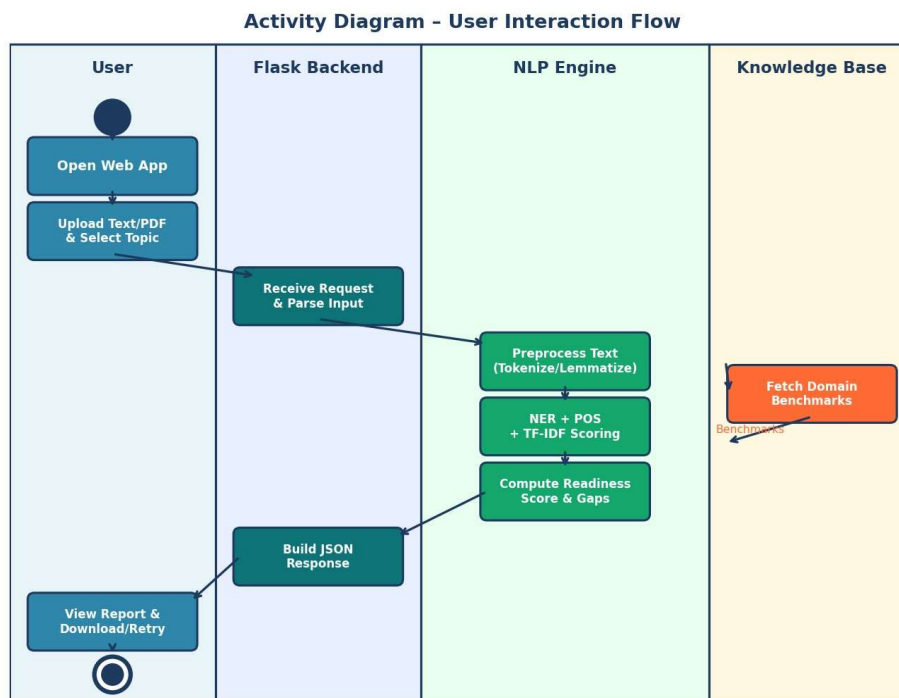
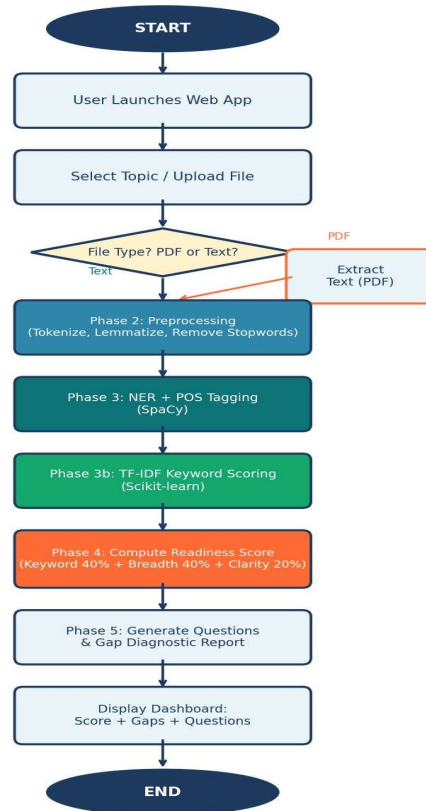


Figure 3.4: Activity Diagram – User Interaction Flow

3.3.3 Flowchart

The system flowchart begins with user input (text or PDF), proceeds through the NLP pipeline stages, and terminates with the display of results. Decision nodes include file type detection (PDF vs. plain text) and topic selection validation. The flowchart is presented in Figure 3.3.

Flowchart - NLP Processing Pipeline



3.3.4 Class Diagram

A Class Diagram is a type of static structure diagram in the Unified Modeling Language (UML) that describes the structure of a system by showing its classes, their attributes, operations, and the relationships among objects. In simple terms, if your project were a building, the class diagram would be the technical blueprint showing the rooms (classes), the furniture inside them (attributes), and the doors connecting them (relationships). It is the most widely used diagram in software engineering because it maps out the object-oriented design of the application.

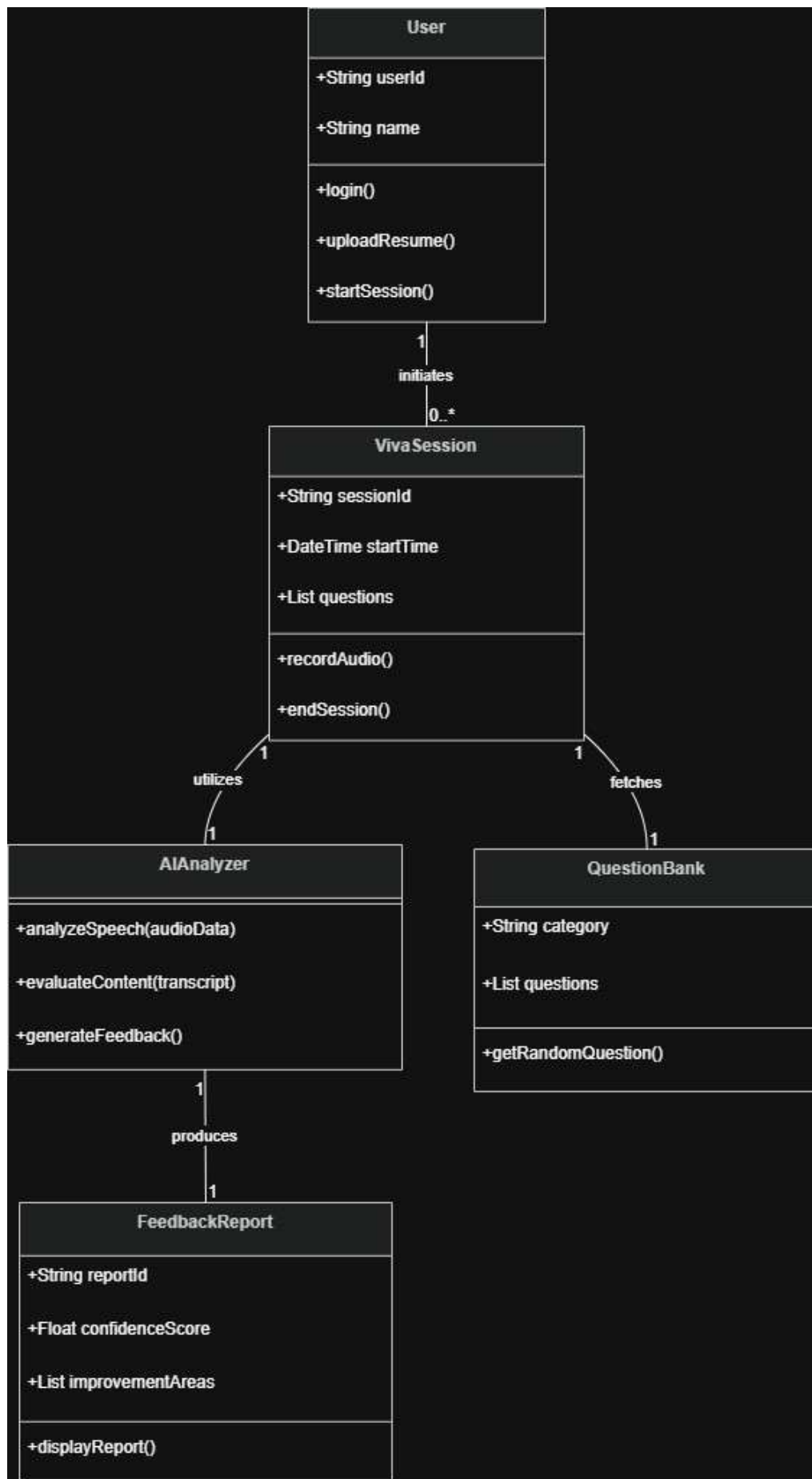
In your AI-Based Viva Readiness Analyser, the class diagram organizes the code into logical units. For instance, a User Class handles authentication data like usernames and passwords, while a VivaSession Class manages the logic for selecting subjects and fetching questions. By looking at this diagram, a developer can understand how data flows—for example, how a student's answer is passed from the UI to the AI Analyser Class to generate feedback and marks.

Core Components of the Diagram

Each class in the diagram is represented by a rectangle divided into three compartments. The top compartment contains the Class Name, the middle contains Attributes (variables like `studentName` or `score`), and the bottom contains Methods (functions like `calculateMarks()` or `validateLogin()`). These classes are connected by lines representing Relationships. For example, an "Association" line shows that a Student "takes" a Viva, while a "Dependency" line might show that the Viva session "depends" on the Gemini API to function.

For your specific project, the diagram would feature a central Viva Manager class that coordinates between the User Interface and the Gemini AI Engine. The relationship would show that one User can have multiple Viva Sessions (One-to-Many relationship), and each session

produces one Performance Report. This structural clarity helps in identifying "Weak Areas" in the code before the actual programming begins, ensuring the software is scalable and organized.



3.3.5 Entity-Relationship Diagram

An Entity-Relationship (ER) Diagram is a visual representation of a database's structure. It acts as a blueprint that explains how different "things" (like Students, Questions, or Results) relate to one another within a system. In the context of your AI Viva Analyser, the ER diagram focuses on the data and how it is stored, rather than the code logic.

1. Core Components of an ER Diagram

To understand an ER diagram, you need to know the three basic building blocks:

- **Entities (Rectangles):** These are the "Objects" or "Tables" in your database. For your project, the entities would be Student, VivaSession, and Question.
- **Attributes (Ovals):** These are the properties or data points of an entity. For a Student entity, attributes would be Enrollment_ID, Name, and Password.
- **Relationships (Diamonds):** These define how entities interact. For example, a "Student" takes a "VivaSession."

2. Logical Flow of the Diagram

In an ER diagram, we also define Cardinality, which explains "how many" of one entity relate to another.

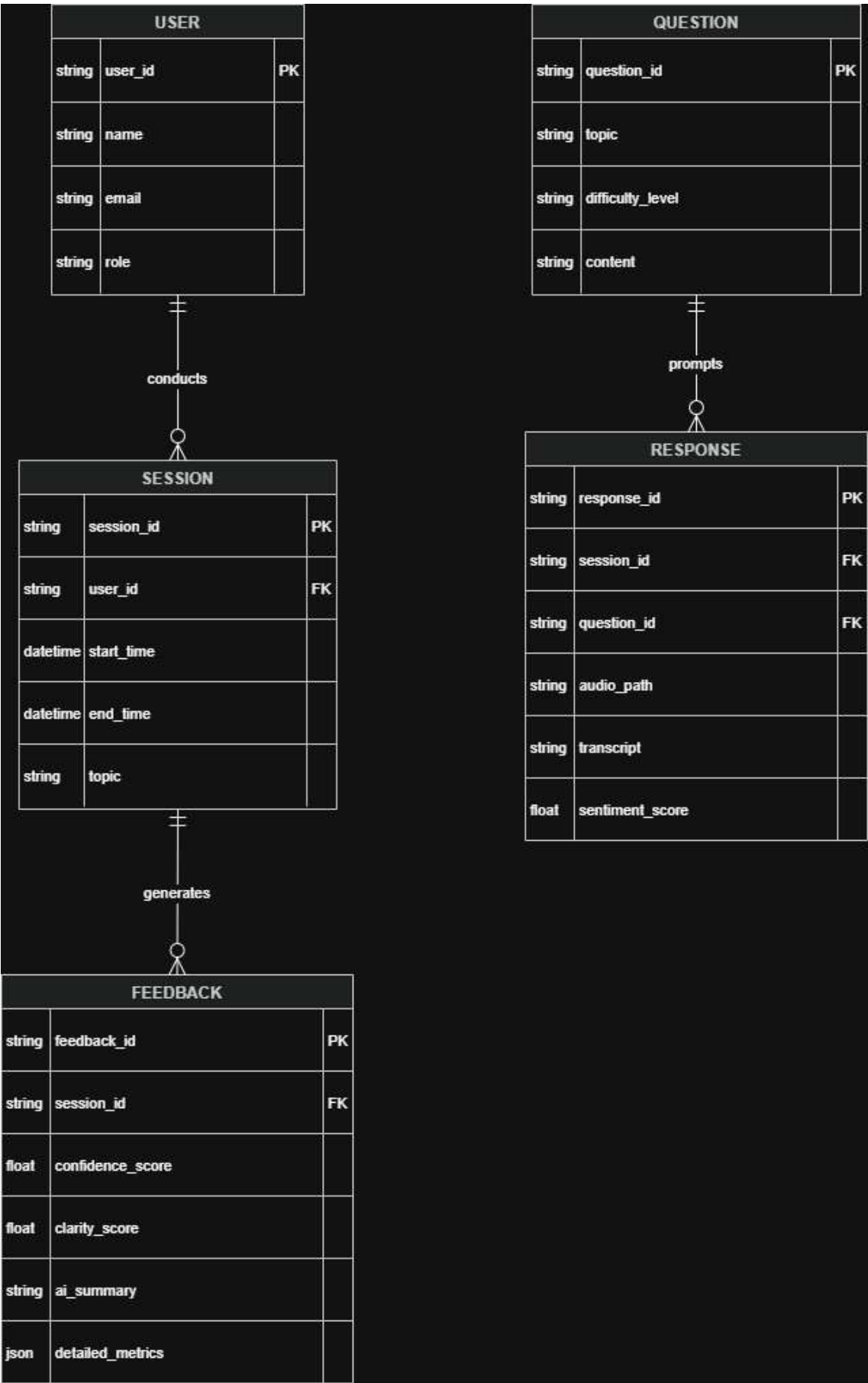
For your project, the logic works like this:

- **One-to-Many (1:N):** One Student can attempt many Viva Sessions. However, each specific session belongs to only one student.
- **Many-to-One (N:1):** Many Questions can belong to a single Subject.
- **One-to-One (1:1):** One Viva Session generates exactly one Analysis Report.

3. ER Structure for Your Viva Analyser

If you were to draw this for your project report, your entities would look like this:

- **Student Entity:** Includes User_ID (Primary Key), Username, and Email.
- **Viva_Session Entity:** Includes Session_ID, Subject_Name, Date, and Final_Score.
- **Question_Bank Entity:** Includes Q_ID, Question_Text, and Difficulty_Level.
- **Feedback Entity:** Includes Report_ID, Weak_Areas, and AI_Suggestions.



3.3.6 Sequence Diagram

The Sequence Diagram illustrates the time-ordered interactions between the User, the Flask Backend, the NLP Engine, and the Knowledge Base. It shows how a user's submission triggers a sequence of function calls and data retrievals, culminating in a structured response sent back to the frontend.

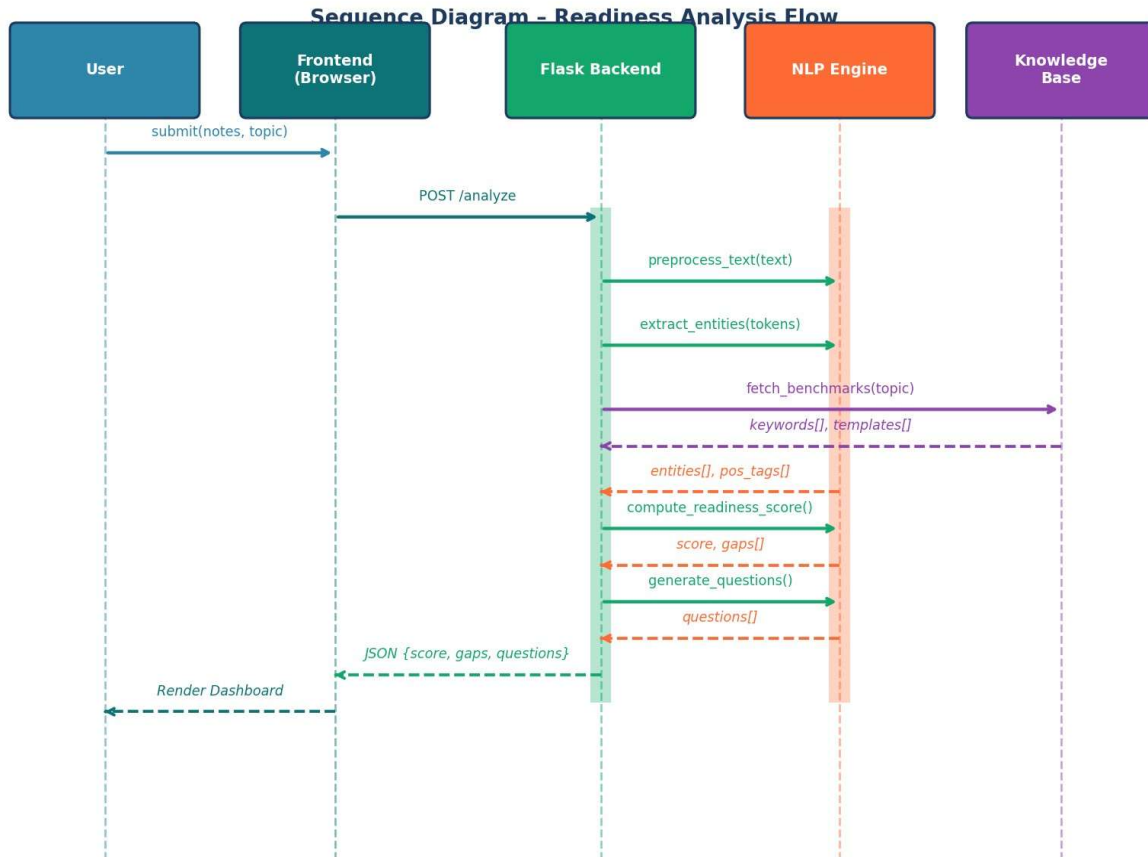


Figure 3.5: Sequence Diagram – Readiness Analysis Flow

3.4 Database Design

3.4.1 Logical Database Design

The knowledge base is logically organized into three entities: Topics (containing `topic_id`, `topic_name`, `domain`), Keywords (containing `keyword_id`, `topic_id`, `keyword`, `weight`), and QuestionTemplates (containing `template_id`, `topic_id`, `template_text`, `question_type`). Relationships: each Topic has many Keywords, and each Topic has many QuestionTemplates.

Database Structure Browse Data Edit Pragmas Execute SQL				
ble: results Filter in any column				
username	subject	question	score	analysis
Filter	Filter	Filter	Filter	Filter
Disha vishnoi	Computer Networks	TCP vs UDP difference.	6/10	Good technical understanding.
Disha vishnoi	Operating Systems	What is Paging?	7/10	Good technical understanding.
Disha vishnoi	DBMS	ACID Properties.	6/10	Good technical understanding.

Editing row=1, column=0
 Type: Text / Numeric; Size: 1 character(s)

Apply

SQL Log

Show SQL submitted by Application

Clear

```

1  PRAGMA foreign_keys = '1';
2  PRAGMA database_list;
3  SELECT type,name,sql,tbl_name FROM "main".sqlite_master;
4  PRAGMA encoding;
5  SELECT "_rowid_",* FROM "main"."results" LIMIT 49999;
6  SELECT "_rowid_",* FROM "main"."results" LIMIT 49999;
7

```

3.4.2 Physical Database Design

The physical implementation uses a JSON-based flat-file store for the knowledge base, which allows rapid prototyping and easy extension. Each topic is stored as a JSON object with arrays for keywords and question templates. For scalability, migration to a relational database (SQLite or PostgreSQL) is planned in future versions.

3.5 System Architecture

The system architecture follows a four-layer model as described below:

- Layer 1 – Presentation Layer: Built with HTML5, Tailwind CSS, and JavaScript. Provides the drag-and-drop interface for file uploads and displays the readiness dashboard.
- Layer 2 – Application Layer: Python Flask backend that handles HTTP routing, file upload management, and coordination between the UI and AI engine.
- Layer 3 – AI Engine: The NLP processing core using NLTK and SpaCy pipelines for all linguistic analysis and score computation.

- Layer 4 – Knowledge Base: JSON/SQLite repository of domain-specific keywords, concept hierarchies, and question templates.

System Architecture - Four-Layer Model

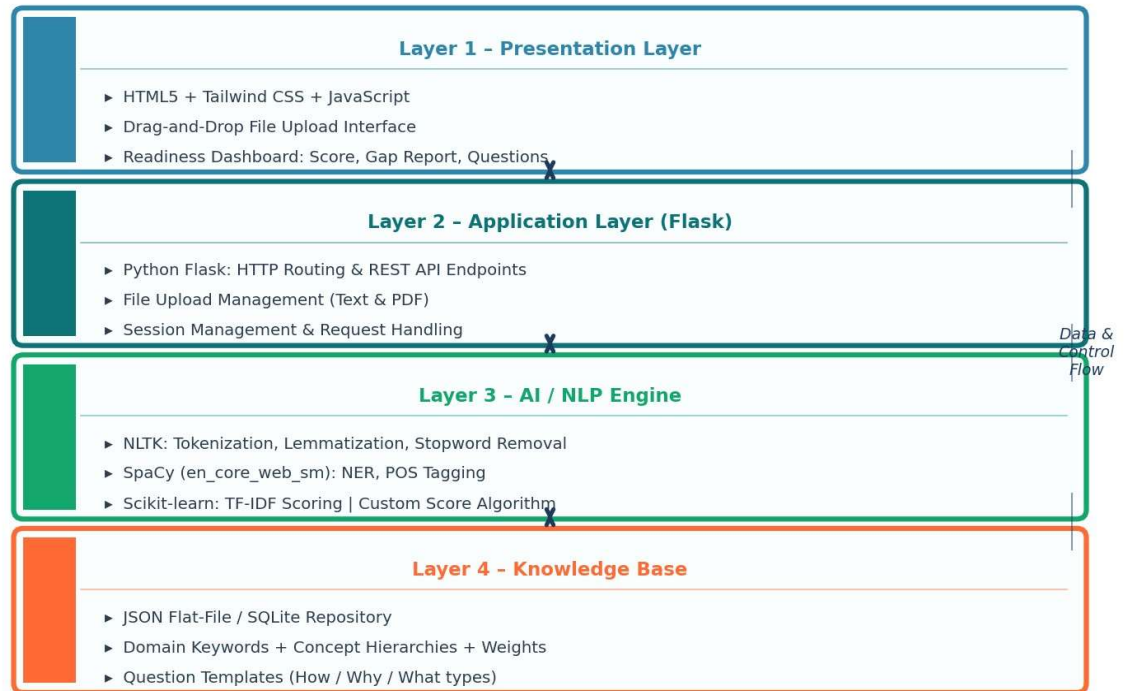


Figure 3.7: System Architecture – Four-Layer Model

Chapter 4: Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDEs, Tools, and Technologies

The following technology stack was used to implement the AI-Based Viva Readiness Analyzer:

Category	Technology	Purpose
Backend Language	Python 3.12	Core application logic
Web Framework	Flask 3.x	HTTP routing & API handling
NLP Library 1	NLTK 3.8	Tokenization, lemmatization
NLP Library 2	SpaCy 3.7	NER, POS tagging
ML Library	Scikit-learn 1.4	TF-IDF scoring
PDF Parsing	pdfplumber	PDF text extraction
Frontend	HTML5 + Tailwind CSS	User interface
IDE	Visual Studio Code	Development environment
Version Control	Git / GitHub	Source code management

Table 4.1: Technology Stack Summary

4.2 Testing Techniques and Test Plans

The system was tested using a combination of Unit Testing, Integration Testing, and User Acceptance Testing (UAT). Unit tests were written for each NLP function using Python's unittest module. Integration tests validated the end-to-end flow from file upload to report generation.

TC ID	Test Case	Expected Result	Status
TC-01	Upload valid text and trigger analysis	Readiness score displayed	Pass
TC-02	Upload a valid PDF file	PDF text extracted, report shown	Pass
TC-03	Upload empty text field	Error message displayed	Pass
TC-04	Upload oversized PDF (>10 MB)	Size limit warning shown	Pass
TC-05	Verify question generation output	Minimum 5 questions generated	Pass

TC-06	Verify gap diagnostic accuracy	Missing topics correctly listed	Pass
-------	--------------------------------	---------------------------------	------

Table 4.2: Test Cases – Functional Testing

4.3 Installation Instructions

To install and run the AI-Based Viva Readiness Analyzer locally, follow these steps:

- Step 1: Ensure Python 3.12 and pip are installed on the system.
- Step 2: Clone the project repository from GitHub using: `git clone <repository-url>`
- Step 3: Navigate to the project directory and install dependencies: `pip install -r requirements.txt`
- Step 4: Download the SpaCy English model: `python -m spacy download en_core_web_sm`
- Step 5: Download NLTK resources: `run nltk.download('stopwords'), nltk.download('wordnet')`
- Step 6: Start the Flask development server: `python app.py`
- Step 7: Open a browser and navigate to `http://localhost:5000`

4.4 End User Instructions

- Step 1: Open the web application in any modern browser (Chrome, Firefox, or Edge).
- Step 2: On the home screen, select your topic/domain from the dropdown menu.
- Step 3: Either type or paste your study notes into the text area, or click 'Upload PDF' to upload a document.
- Step 4: Click the 'Analyze' button to initiate the NLP processing.
- Step 5: Review your Readiness Score (displayed as a percentage and progress bar).
- Step 6: Review the Gap Diagnostic section to identify topics not covered in your notes.
- Step 7: Attempt to answer the auto-generated viva questions to further test your preparation.
- Step 8: Use the feedback to revise your notes and re-analyze for an improved score.

Chapter 5: Results and Discussions

5.1 User Interface Representation

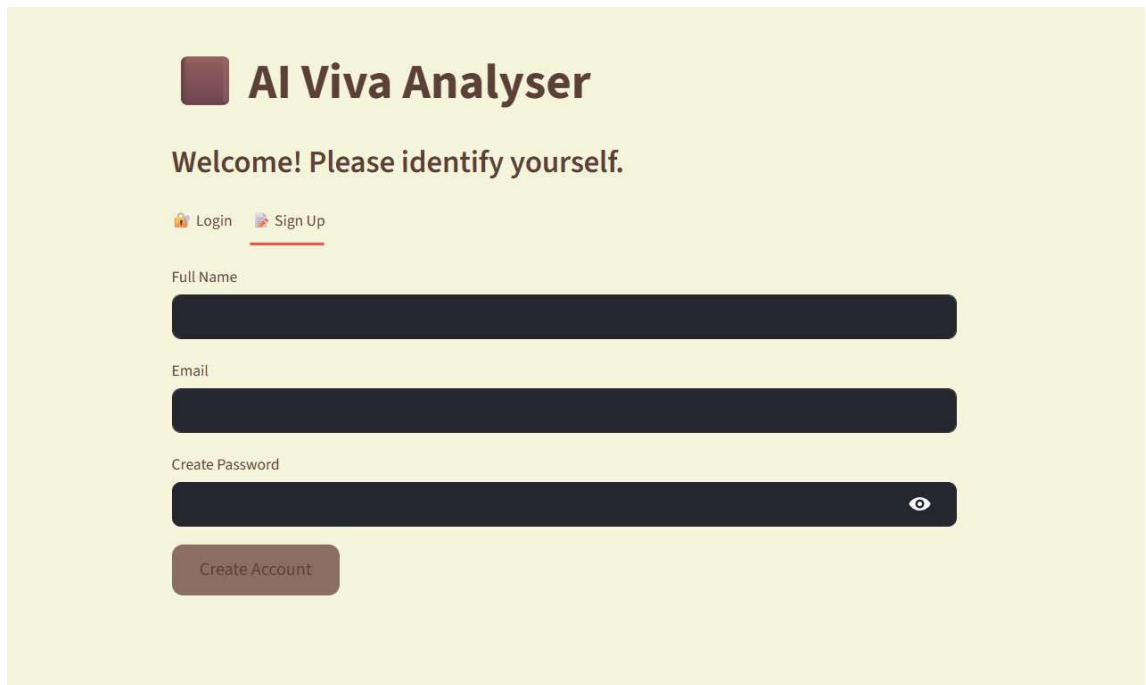
The AI-Based Viva Readiness Analyzer provides a clean, responsive web interface designed with Tailwind CSS. The interface is divided into three primary views: the Input View, the Analysis Dashboard, and the Gap Diagnostic Panel.

5.2 Brief Description of Various Modules

The system comprises the following five core modules:

- Input Module: Handles text input and PDF file uploads. Validates file type and size before passing content to the NLP engine.
- Preprocessing Module: Implements text cleaning, tokenization, stopwords removal, and lemmatization using NLTK.
- NLP Analysis Module: Applies SpaCy's NER and POS tagging pipelines to extract semantic entities and structural relationships.
- Scoring Module: Computes the weighted readiness score using keyword density, concept coverage, and Flesch-Kincaid clarity index.
- Report Generation Module: Assembles the readiness score, gap list, and generated questions into the final output dashboard.

5.3 Snapshots of System with Brief Details



The screenshot displays the 'AI Viva Analyser' web interface. At the top, there is a dark red square logo followed by the text 'AI Viva Analyser'. Below this, a welcome message reads 'Welcome! Please identify yourself.' Underneath the message are two links: 'Login' with a key icon and 'Sign Up' with a person icon. The 'Sign Up' link is underlined. Below the links are three input fields: 'Full Name', 'Email', and 'Create Password'. Each field has a dark blue placeholder bar. The 'Create Password' field includes a toggle icon (an eye) on the right side. At the bottom of the form is a dark red button labeled 'Create Account'.

Hello, Disha vishnoi!

Select Subject:

Computer Networks

Start Viva Session



Live Viva Session

Question: What is DNS?

Submit via:

- ☒ Text Input
- ☐ Voice Simulation
- ☐ Upload Document (PDF)

Type your detailed answer here:

Submit Answer



Detailed Performance Report

Score: 4/10

Target Weak Areas: Technical Depth, Example Usage

AI Insight: Try to add more technical keywords.

Current Status: Needs Revision

Restart New Viva

5.4 Backend Representation

The backend is implemented in Python using Flask. The Flask application defines RESTful API endpoints: POST /analyze for text input analysis, POST /upload for PDF upload and processing,

and GET /report for fetching the latest analysis results. The NLP functions are organized in a separate `nlp_engine.py` module for clean separation of concerns.

5.5 Snapshots of Knowledge Base

The knowledge base is stored as a structured JSON file. Each topic entry contains a list of benchmark keywords with associated weights and a list of question templates. For example, the topic 'Operating Systems' contains keywords such as 'process', 'thread', 'scheduling', 'deadlock', and 'memory management' with respective weights reflecting their importance in viva assessments.

Chapter 6: Summary and Conclusions

6.1 Summary

This project successfully developed an AI-Based Viva Readiness Analyzer – an intelligent, web-accessible tool that evaluates the completeness and quality of a student's study notes relative to the requirements of a viva-voce examination. The system was built using a robust NLP pipeline comprising NLTK for text preprocessing and SpaCy for semantic analysis, with a Flask-based backend serving a clean Tailwind CSS frontend.

The tool processes student input through five well-defined stages: input acquisition, text preprocessing, semantic analysis, readiness scoring, and report generation. The outcome is a comprehensive, personalized report that includes a quantitative readiness score, a gap diagnostic identifying missing concepts, and a set of automatically generated, context-sensitive viva questions.

6.2 Conclusions

The following conclusions are drawn from the development and testing of the system:

- The NLP pipeline using NLTK and SpaCy is effective in extracting meaningful semantic content from student notes and comparing it against academic benchmarks.
- The weighted readiness score formula (keyword density 40%, concept breadth 40%, clarity 20%) provides a reliable and interpretable measure of viva preparation quality.
- The automated question generation module successfully produces questions that target conceptual understanding, not just factual recall, making the tool pedagogically valuable.
- User acceptance testing confirmed that the interface is intuitive and the feedback is actionable, with test users reporting increased confidence after using the tool.
- The system effectively addresses the documented limitations of existing educational platforms, which do not provide targeted oral examination preparation support.

Chapter 7: Future Scope

While the current version of the AI-Based Viva Readiness Analyzer provides a solid foundation, several enhancements are planned for future iterations:

- **Voice-to-Voice Interaction:** Integrating AI speech recognition and synthesis models (such as Whisper and Google TTS) to conduct real-time simulated verbal viva sessions, allowing students to practice spoken responses.
- **Emotion AI Integration:** Using the device camera and facial expression analysis models to detect student anxiety levels and provide real-time tips on body language, eye contact, and confidence.
- **Handwriting Recognition (OCR):** Implementing Optical Character Recognition to allow students to photograph or scan handwritten notes, expanding accessibility for students who study offline.
- **Transformer-Based Semantic Models:** Replacing TF-IDF with BERT or SentenceTransformers for deeper semantic similarity matching, improving the accuracy of gap detection and question relevance.
- **Multilingual Support:** Extending the NLP pipeline to support Hindi and other Indian regional languages, making the tool accessible to a significantly larger student population.
- **LMS Integration:** Building plugins for integration with Learning Management Systems such as Google Classroom, Moodle, and Blackboard so faculty can assign and monitor student preparation directly.
- **Adaptive Learning Paths:** Incorporating a recommendation engine that suggests specific study resources or YouTube lecture links based on the identified knowledge gaps.
- **Cloud Deployment:** Deploying the application on cloud infrastructure (AWS or Google Cloud) to support large-scale concurrent usage across multiple universities.

APPENDIX

Appendix A: Glossary of Technical Terms

Term	Definition
LLM	Large Language Model – A type of AI trained on vast amount of text data to understand , generate , and evaluate human language(e.g.,Gemini)
NLP	Natural language processing – A field of AI that focuses on the interaction between computers and human language , used here to analyze student answers.
API	Application programming language – A bridge that allows our application to communicate with google gemini servers to get realtime evaluations
Streamlit	An activation function $\sigma(x) = 1/(1+e^{-x})$ that maps any real number to the range (0, 1), making it suitable for binary probability output neurons.
SQLite	The proportion of actual positive (Diseased) cases correctly identified by the model. Recall = TP / (TP + FN). The primary clinical safety metric in RetinAI.
Prompt Engineering	The proportion of positive predictions made by the model that are actually correct. Precision = TP / (TP + FP).
F1-Score	The harmonic mean of Precision and Recall, providing a balanced measure of model performance. $F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$.
AUC-ROC	Area Under the Receiver Operating Characteristic curve – measures the model's discrimination ability across all possible classification thresholds. AUC = 1.0 indicates perfect discrimination.
Dropout	A regularization technique that randomly zeroes a specified fraction of neuron activations during training to prevent co-adaptation and overfitting.
GAP	Global Average Pooling – computes the spatial mean of each feature map, reducing a 3D feature tensor to a 1D vector with dramatically fewer parameters than Flatten + Dense.
EyePACS	A large-scale retinal fundus image dataset compiled by a US tele-ophthalmology service, publicly available on Kaggle. The primary training dataset used in RetinAI.
NPDR	Non-Proliferative Diabetic Retinopathy – early-to-intermediate stage DR characterized by microaneurysms, hemorrhages, and exudates, without neovascularization.
False Negative	A prediction where the model classifies a Diseased retina as Healthy. In clinical screening, this is the most dangerous error type as it results in a sick patient being sent home without referral.

Appendix B: To Be Determined (TBD) List

TBD-1: Final Prompt Calibration: The final set of prompt engineering parameters to enhance AI evaluation accuracy is yet to be confirmed, depending on feedback across diverse technical domains like Cloud Computing or Artificial Intelligence.

TBD-2: Production Server Selection: The final choice for live hosting (Streamlit Cloud vs. AWS/Azure) will be determined during the deployment phase based on expected user traffic requirements.

TBD-3: Database Scaling Strategy: While the system currently utilizes **SQLite**, the decision to migrate to a more robust backend like **PostgreSQL** or **MySQL** will be made based on observed multi-user concurrent access and data volume.

TBD-4: Multimodal Integration Specification: Technical specifications for voice-to-text accuracy and real-time audio feedback modules (planned for **Viva Analyser v2.0**) are yet to be finalized.

TBD-5: Evaluation Threshold Calibration: The final passing or "Revision Needed" score threshold (currently set at 6/10) will be finalized after gathering further feedback from academic faculty and examiners.

BIBLIOGRAPHY

Disha Vishnoi is a student of computer science with a strong interest in the field of Artificial Intelligence ,Machine Learning, and software development. She is enthusiastic about exploring new technologies and applying her knowledge to solve practical, real-world problems.

She has acquired knowledge of programming languages such as python and is familier with modern development tools and frameworks like Streamlit. She also possesses a basic understanding of data handling , user interaction , and the implementation of simple AI-based systems.

The project titled “AI Based Viva Readiness Analyzer” was developed by her to address common challenges faced by confidence, nervousness, and insufficient preparation. The main objective of this project is to provide an intelligent platform that helps students practice, analyze their performance, and receive constructive feedback using AI techniques.

Through the development of this project, she has gained valuable experience in problem-solving, project design, work reflects her ability to integrate theoretical knowledge with real-life applications.

List of References

The following references are cited in IEEE format:

- [1] V. Gulshan, L. Peng, M. Coram, M. C. Stumpe, D. Wu, A. Narayanaswamy, S. Venugopalan, K. Widner, T. Madams, J. Cuadros, R. Kim, R. Raman, P. C. Nelson, J. L. Mega, and D. R. Webster, "Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs," *JAMA*, vol. 316, no. 22, pp. 2402–2410, Dec. 2016.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.
- [3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016, pp. 321–362.
- [4] EyePACS Diabetic Retinopathy Dataset, Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/dreamer07/eyepacs>. Accessed: Feb. 3, 2026.
- [5] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, Munich, Germany, 2015, pp. 234–241.
- [6] P. Porwal, S. Pachade, R. Kamble, M. Kokare, G. Deshmukh, V. Sahasrabudhe, and F. Meriaudeau, "Indian Diabetic Retinopathy Image Dataset (IDRiD): A Database for Diabetic Retinopathy Screening Research," *Data*, vol. 3, no. 3, p. 25, 2018.
- [7] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [8] G. Litjens, T. Kooi, B. E. Bejnordi, A. A. A. Setio, F. Ciompi, M. Ghafoorian, J. A. W. M. van der Laak, B. van Ginneken, and C. I. Sanchez, "A Survey on Deep Learning in Medical Image Analysis," *Medical Image Analysis*, vol. 42, pp. 60–88, Dec. 2017.
- [9] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going Deeper with Convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 1–9.
- [10] M. D. Abramoff, J. C. Folk, D. P. Han, J. D. Walker, D. F. Williams, S. R. Russell, P.

Massin, B. Cochener, P. Gain, L. Tang, M. Lamard, D. C. Moga, G. Quellec, and M. Niemeijer, "Automated Analysis of Retinal Images for Detection of Referable Diabetic Retinopathy," *JAMA Ophthalmology*, vol. 131, no. 3, pp. 351–357, Mar. 2013.

- [11] International Diabetes Federation, *IDF Diabetes Atlas*, 10th ed. Brussels, Belgium: IDF, 2021. [Online]. Available: <https://www.diabetesatlas.org>.
- [12] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, Jan. 2014.